KANT AX/AI 엔지니어 트랙 · 35

Docker 5

명령어 한눈에 보기

29부터 배운 docker 명령과
오늘 수업 명령을 한 장에 모아요

```
~/kant-shop $ docker run -d --name my-pg \
-e POSTGRES_PASSWORD=admin1234 \
-p 5432:5432 postgres:18
48d1c194469c97d81d48c27d4b50f3a34ea926ed98d99eb8c4245d057a0dc59f

~/kant-shop $ docker cp kant_shop.sql my-pg:/tmp/kant_shop.sql
Successfully copied 610B (transferred 2.56kB) to my-pg:/tmp/kant_shop.sql

~/kant-shop $ docker exec -i my-pg psql -U postgres < kant_shop.sql
DROP DATABASE
CREATE DATABASE
You are now connected to database "kant_shop" as user "postgres".
CREATE TABLE
INSERT 0 5
```

오늘의 순서

1	**명령 읽는 법** docker 다음 할 일, 옵션, 이름 / 옵션 표 / \로 줄 나누기	5–7쪽
2	**이미지와 컨테이너** pull, images, run, ps, stop, start, restart, logs --tail, -f, rm, rmi	8–12쪽
3	**컨테이너 안에서 작업하기** kant_shop.sql, docker cp, exec 세 가지, psql -f, exec -i < 파일, -it 함정	13–18쪽
4	**볼륨** volume create, ls, inspect, -v pgdata:…, volume rm	19–21쪽
5	**YAML과 compose** YAML 모양, 들여쓰기 2칸, 탭 오류, run과 compose.yaml, config, up, logs, down, ls, pull	22–30쪽
6	**오류와 실제 상황** 오류 모음, 수업 흐름 그대로, compose로 다시, 다 쓴 것 정리	31–34쪽

3–4쪽은 들어가기, 35–36쪽은 긴 이름과 --help, 37–38쪽은 치트시트 두 장이에요.

들어가기

docker 명령이 여섯 챕터에 흩어져 있어요

29
Docker 1

- `pull`
- `images`
- `run`
- `ps`
- `stop`
- `start`
- `restart`
- `rm`
- `rmi`
- `logs`

30
Docker 2

- `exec -it`
- `volume ls`
- `volume rm`
- `compose up`
- `compose ps`
- `compose down`

31
nginx

- `exec 이름 명령`
- `compose logs`
- `compose stop`
- `compose start`

33·34
MySQL, PostgreSQL

- `compose exec`
- `-e 환경 변수`

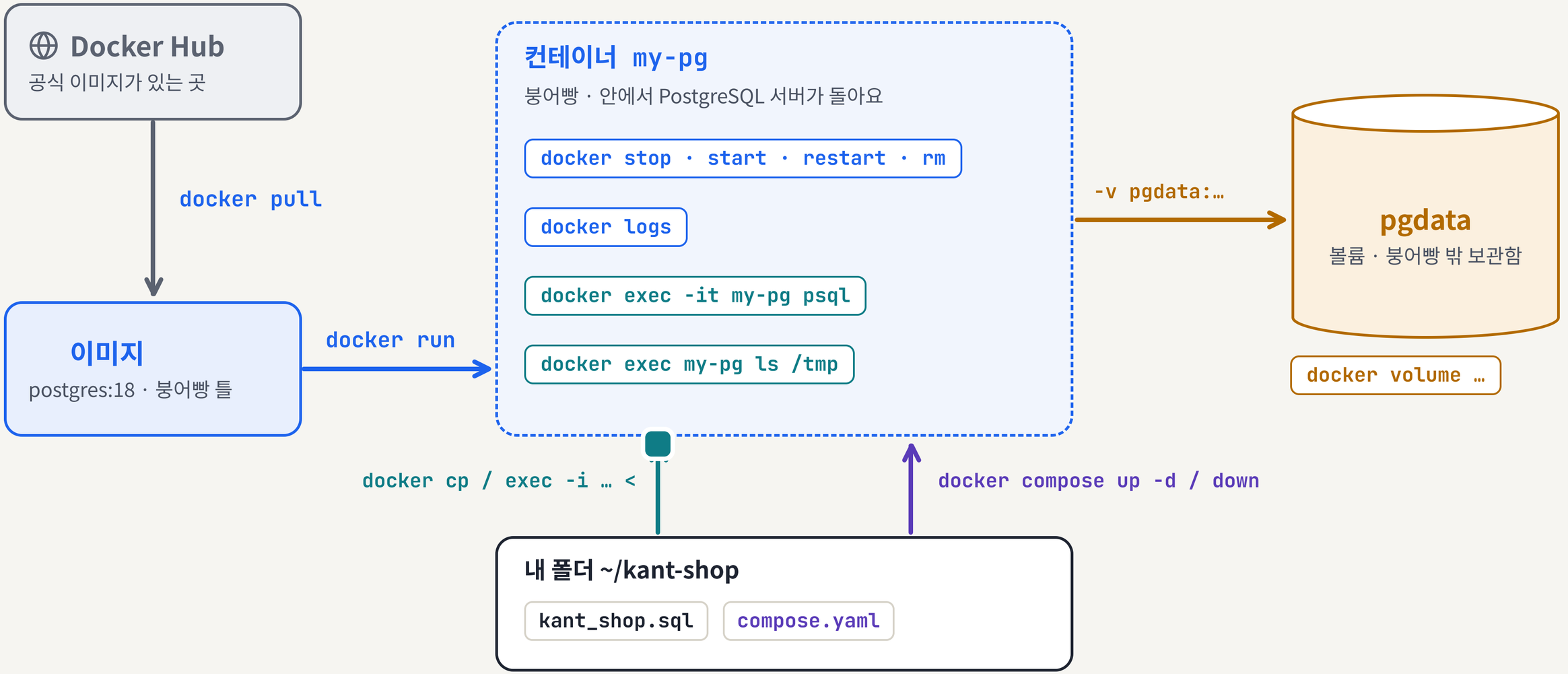
오늘
수업 레포

- `cp`
- `exec -i … <`
- `volume create`
- `volume inspect`
- `compose config`

29~34에서 배운 명령 20개, 오늘 새로 나온 명령 5개

들어가기

명령은 이미지, 컨테이너, 볼륨, 내 폴더 사이를 오가요



명령마다 무엇을 움직이는지 보면 덜 헷갈려요.

1. 명령 읽는 법

docker 다음은 할 일, 그 뒤에 옵션과 이름이 와요

```
docker run -d --name my-pg -e POSTGRES_PASSWORD=admin1234 -p 5432:5432 postgres:18
       |                                                                      |
    할 일                                                                 이미지 이름
```

옵션 네 개

다른 명령도 같은 모양이에요

할 일	옵션	이름	그 뒤에 붙는 것
`docker volume create`		`pgdata`	
`docker exec`	`-it`	`my-pg`	`psql -U postgres`
`docker logs`	`--tail 3`	`my-pg`	
`docker compose up`	`-d`		
`docker cp`		`kant_shop.sql`	`my-pg:/tmp/kant_shop.sql`

할 일이 두 단어일 때도 있어요. volume create, compose up처럼요.

1. 명령 읽는 법

옵션은 글자 하나짜리가 많아요

옵션	뜻	언제	챕터
`-d`	**뒤에서 켜기**	터미널을 계속 쓰고 싶을 때	29
`--name 이름`	**컨테이너 이름 붙이기**	긴 ID 대신 이름으로 부를 때	29
`-p 5432:5432`	**포트 연결 (내 컴퓨터:컨테이너)**	밖에서 접속할 때	29
`-e 이름=값`	**환경 변수 넘기기**	비밀번호를 정할 때	33
`-v pgdata:경로`	**볼륨 연결**	지워도 데이터를 남길 때	30
`-it`	**들어가서 직접 치기**	psql, bash를 열 때	29
`-i`	**파일 내용을 넘기기**	< 파일로 SQL을 넣을 때	오늘
`--tail 3`	**마지막 3줄만**	기록이 길 때	오늘
`-f`	**계속 보기 (Ctrl+C로 멈춤)**	새 기록을 기다릴 때	오늘

주황색 세 줄은 오늘 처음 나와요. 2, 3파트에서 직접 쳐 봐요.

1. 명령 읽는 법

긴 명령은 줄 끝에 \를 붙여 여러 줄로 나눠요

```
~/kant-shop $ docker run -d --name my-pg \
> -e POSTGRES_PASSWORD=admin1234 \
> -p 5432:5432 postgres:18
48d1c194469c97d81d48c27d4b50f3a34ea926ed98d99eb8c4245d057a0dc59f
```

\ 는 "다음 줄도 같은 명령"이라는 표시예요.

Enter를 쳐도 실행하지 않고 > 를 띄워 다음 줄을 기다려요. 한 줄로 친 것과 똑같이 실행돼요.

```
~/kant-shop $ docker run -d --name my-pg \
> -e POSTGRES_PASSWORD=admin1234 \
> -p 5432:5432 postgres:18
docker: invalid reference format

Run 'docker run --help' for more information
bash: line 2: -e: command not found
```

\ 뒤에 띄어쓰기가 있으면 줄이 끊겨요

Mac 터미널과 Windows Git Bash에서 써요. 수업 자료의 두 가지 쓰는 법은 결과가 같아요.

이미지는 pull로 받고 images로 보고 rmi로 지워요

```
~/kant-shop $ docker pull postgres:18
18: Pulling from library/postgres
… 층 15개를 받아요 (줄 15개 줄임)
Digest: sha256:b9f6b8ff03b62f3cfd9d9ba3fe56226a9991c79d3fb46fd46536263bf02be553
Status: Downloaded newer image for postgres:18
docker.io/library/postgres:18

~/kant-shop $ docker images
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
postgres:18    b9f6b8ff03b6        676MB          175MB
```

다 받았어요

`docker pull postgres:18`

받기

`docker images`

받은 목록

`docker rmi postgres:18`

지우기 (쓰는 컨테이너가 없을 때)

태그 18은 PostgreSQL 18이에요. 숫자로 적어 두면 모두 같은 버전을 받아요.

2. 이미지와 컨테이너

run으로 켜고 ps로 켜진 컨테이너를 봐요

```
~/kant-shop $ docker run -d --name my-pg -e POSTGRES_PASSWORD=admin1234 -p 5432:5432 postgres:18
b64cc6816076fd241aa3b34121f80f477313a530b3931baf5b0b2e622c154793

~/kant-shop $ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                    NAMES
b64cc6816076   postgres:18    "docker-entrypoint.s…"   2 seconds ago   Up 2 seconds   0.0.0.0:5432->5432/tcp   my-pg
```

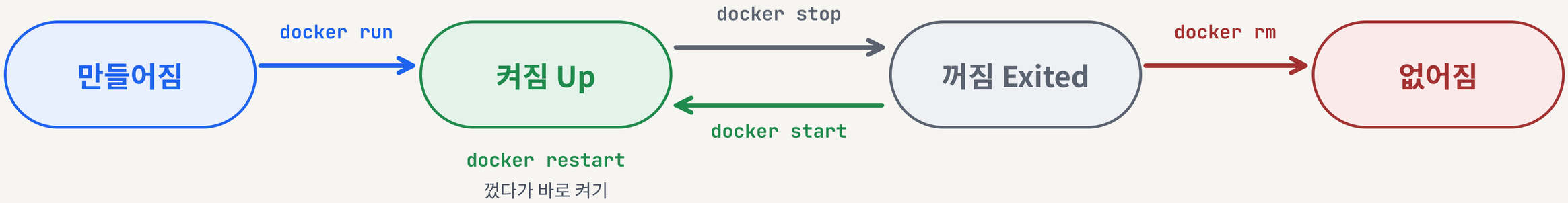
컨테이너 ID
앞 12자리예요

STATUS Up
켜져 있어요

NAMES
--name으로 붙인 이름

ps는 켜진 것만, ps -a는 꺼진 것까지 보여 줘요.

stop, start, restart는 상태만 바꾸고 rm이 지워요




```
~/kant-shop $ docker stop my-pg
my-pg
~/kant-shop $ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                              PORTS     NAMES
b64cc6816076   postgres:18   "docker-entrypoint.s…"   3 seconds ago   Exited (0) Less than a second ago             my-pg

~/kant-shop $ docker start my-pg
my-pg
~/kant-shop $ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS                  PORTS                     NAMES
b64cc6816076   postgres:18   "docker-entrypoint.s…"   14 seconds ago   Up Less than a second   0.0.0.0:5432->5432/tcp   my-pg
```

꺼져도 ps -a에 남아 있어요. 다시 켜면 안의 데이터도 그대로예요.

logs --tail은 마지막 몇 줄만, -f는 새 기록을 계속 보여 줘요

```
~/kant-shop $ docker logs --tail 3 my-pg
… LOG:  checkpoint starting: end-of-recovery immediate wait
… LOG:  checkpoint complete: wrote 0 buffers (0.0%), wrote 3 SLRU buffers; 0 WAL file(s) added, 0 removed, 0
recycled; write=0.001 s, sync=0.001 s, total=0.003 s; sync files=2, longest=0.001 s, average=0.001 s;
distance=0 kB, estimate=0 kB; lsn=0/1774018, redo lsn=0/1774018
… LOG:  database system is ready to accept connections
```

3줄만 나오고 끝나요

```
~/kant-shop $ docker logs -f --tail 1 my-pg
… LOG:  database system is ready to accept connections
^C
```

Ctrl+C를 눌러야 멈춰요

줄 앞의 날짜와 번호는 화면에서 … 로 줄였어요.

Ctrl + C

2. 이미지와 컨테이너

켜진 컨테이너는 못 지우고, 쓰는 이미지도 못 지워요

```
~/kant-shop $ docker rm my-pg
Error response from daemon: cannot remove container "my-pg": container is running: stop the container before
removing or force remove

~/kant-shop $ docker rmi postgres:18
Error response from daemon: conflict: unable to delete postgres:18 (must be forced) - container b64cc6816076 is
using its referenced image b9f6b8ff03b6
```

지울 때는 이 순서예요

1. 끄고

`docker stop my-pg`

2. 컨테이너를 지우고

`docker rm my-pg`

3. 이미지를 지워요

`docker rmi postgres:18`

3. 컨테이너 안에서 작업하기

수업 파일 kant_shop.sql은 데이터베이스, 표, 상품 다섯 개를 만들어요

kant_shop.sql

```
1  -- 1. 데이터베이스 생성
2  DROP DATABASE IF EXISTS kant_shop;
3
4  CREATE DATABASE kant_shop;
5
6  -- 2. 새로 만든 데이터베이스에 접속 (특정 DB를 사용하겠다.)
7  \c kant_shop
8
9  -- 3. 상품 테이블 생성
10 CREATE TABLE product
11 (
12     id SERIAL PRIMARY KEY,
13     name VARCHAR(100) NOT NULL,
14     price INT NOT NULL,
15     stock INT NOT NULL
16 );
17
18 -- 4. 상품 초기 데이터 입력
19 INSERT INTO product (name, price, stock) VALUES
20 ('무선 이어폰', 35000, 100),
21 ('게이밍 마우스', 28000, 85),
22 ('27인치 모니터', 259000, 15),
23 ('USB-C 허브', 19900, 200),
24 ('노트북 거치대', 24000, 7);
```

`-- 뒤는 설명이에요`

주석이라 실행하지 않아요

`IF EXISTS`

있으면 지우고, 없으면 넘어가요

`\c kant_shop`

psql 명령이에요. 이 데이터베이스로 옮겨요

`SERIAL · NOT NULL`

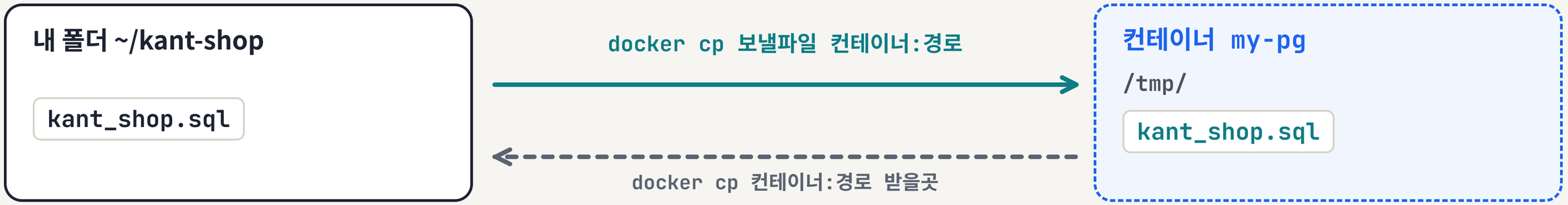
id는 1, 2, 3 자동 번호, 나머지는 비울 수 없어요

`INSERT … VALUES`

상품 다섯 줄을 한 번에 넣어요

3. 컨테이너 안에서 작업하기

docker cp는 내 폴더와 컨테이너 사이에서 파일을 복사해요




```
~/kant-shop $ docker exec my-pg ls /tmp
~/kant-shop $ docker cp kant_shop.sql my-pg:/tmp/kant_shop.sql
Successfully copied 610B (transferred 2.56kB) to my-pg:/tmp/kant_shop.sql
~/kant-shop $ docker exec my-pg ls /tmp
kant_shop.sql

~/kant-shop $ docker cp my-pg:/tmp/kant_shop.sql ./copy.sql
Successfully copied 610B (transferred 2.56kB) to /home/kim/kant-shop/copy.sql
~/kant-shop $ ls
copy.sql  kant_shop.sql
```

/tmp가 비어 있어요

들어갔어요

Windows Git Bash는 30에서 본 export MSYS_NO_PATHCONV=1을 먼저 쳐요. :/tmp 경로가 바뀌지 않아요.

3. 컨테이너 안에서 작업하기

컨테이너 안을 보는 방법은 세 가지예요

```
~/kant-shop $ docker exec -it my-pg bash
root@0f29903380a9:/# ls /tmp
kant_shop.sql
root@0f29903380a9:/# exit
exit

~/kant-shop $ docker exec my-pg ls /tmp
kant_shop.sql

~/kant-shop $ docker exec -it my-pg psql -U postgres -d kant_shop
kant_shop=# \dt
           List of tables
 Schema |  Name   | Type  |  Owner
--------+---------+-------+----------
 public | product | table | postgres
(1 row)
```

`docker exec -it 이름 bash`

들어가서 여러 명령을 쳐요. exit로 나와요.

`docker exec 이름 명령`

명령 하나만 실행하고 바로 끝나요.

`docker exec -it 이름 psql …`

psql을 바로 열어요. \q로 나와요.

3. 컨테이너 안에서 작업하기

방법 1. 복사한 파일을 psql -f로 실행해요

```
~/kant-shop $ docker exec -it my-pg psql -U postgres -f /tmp/kant_shop.sql
psql:/tmp/kant_shop.sql:2: NOTICE:  database "kant_shop" does not exist, skipping
DROP DATABASE
CREATE DATABASE
You are now connected to database "kant_shop" as user "postgres".
CREATE TABLE
INSERT 0 5

~/kant_shop $ docker exec -it my-pg psql -U postgres -d kant_shop -c "SELECT * FROM product"
 id |     name      | price  | stock
----+---------------+--------+-------
  1 | 무선 이어폰   |  35000 |   100
  2 | 게이밍 마우스 |  28000 |    85
  3 | 27인치 모니터 | 259000 |    15
  4 | USB-C 허브    |  19900 |   200
  5 | 노트북 거치대 |  24000 |     7
(5 rows)
```

IF EXISTS: 없어서 넘어가요

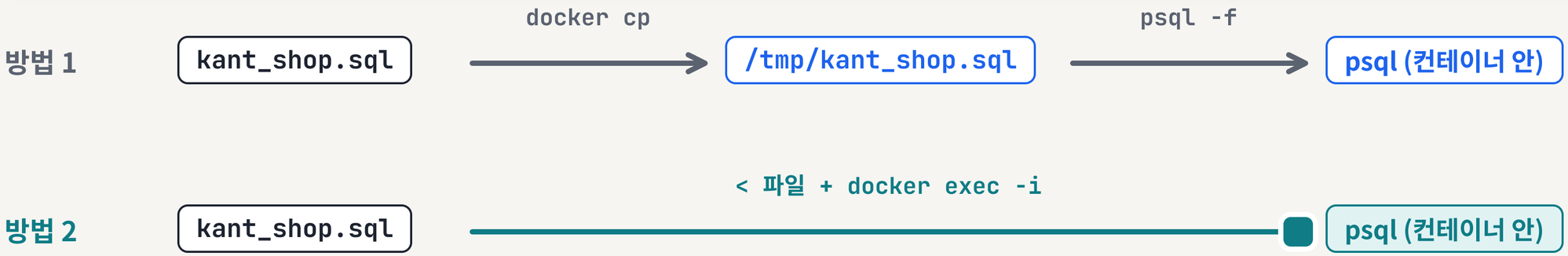
\c kant_shop 줄

상품 다섯 줄

-f 파일은 파일 안의 SQL을 위에서부터 차례로 실행해요. 파일은 컨테이너 안에 있어야 해요.

방법 2. 내 폴더의 파일을 < 로 바로 넘겨요

```
~/kant-shop $ docker exec -i my-pg psql -U postgres < kant_shop.sql
DROP DATABASE
CREATE DATABASE
You are now connected to database "kant_shop" as user "postgres".
CREATE TABLE
INSERT 0 5
```



< 는 파일 내용을 명령에 흘려 넣어요. -i가 그 내용을 컨테이너 안까지 이어 줘요. 복사할 필요가 없어요.

3. 컨테이너 안에서 작업하기

< 로 넘길 때 -it를 쓰면 거절돼요

```
~/kant-shop $ docker exec -it my-pg psql -U postgres < kant_shop.sql
cannot attach stdin to a TTY-enabled container because stdin is not a terminal

~/kant-shop $ docker exec -i my-pg psql -U postgres < kant_shop.sql
DROP DATABASE
CREATE DATABASE
…
```

`-i`	**입력을 이어요**	키보드든 파일이든 받아요
`-t`	**화면(터미널)을 이어요**	키보드로 칠 때만 써요

직접 칠 때는 -it, 파일을 넘길 때는 -i예요.

4. 볼륨

볼륨은 create로 만들고 ls로 보고 inspect로 자세히 봐요

```
~/kant-shop $ docker volume create pgdata
pgdata
~/kant-shop $ docker volume ls
DRIVER    VOLUME NAME
local     pgdata
~/kant-shop $ docker volume inspect pgdata
[
    {
        "CreatedAt": "2026-11-13T04:44:54Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/pgdata/_data",
        "Name": "pgdata",
        "Options": null,
        "Scope": "local"
    }
]
```

이름

`docker volume create pgdata`

이름 붙여 만들기

`docker volume ls`

볼륨 목록

`docker volume inspect pgdata`

자세히 보기 (어디에 저장되는지)

`docker volume rm pgdata`

지우기 (쓰는 컨테이너가 없을 때)

Mountpoint는 Docker가 데이터를 두는 곳이에요. 직접 열지 않고 컨테이너로 읽어요.

4. 볼륨

볼륨을 붙이면 컨테이너를 지워도 상품이 남아요

```
~/kant-shop $ docker run -d --name my-pg -e POSTGRES_PASSWORD=admin1234 -p 5432:5432 -v pgdata:/var/lib/postgresql postgres:18
1bb2985a53570708f7efc19cf0f1bd50e022dc9153fc14ce194908bf1cde8b62
~/kant-shop $ docker exec -i my-pg psql -U postgres < kant_shop.sql
… 결과 6줄 줄임 (마지막 줄 INSERT 0 5)
~/kant-shop $ docker stop my-pg
my-pg
~/kant-shop $ docker rm my-pg ● 컨테이너를 지웠어요
my-pg
~/kant-shop $ docker run -d --name my-pg -e POSTGRES_PASSWORD=admin1234 -p 5432:5432 -v pgdata:/var/lib/postgresql postgres:18
715280d2bf1fb5ddcdd27f164e05901cabe17fc40b4f325114d1f335767480ec ● 같은 볼륨으로 새로 켰어요
~/kant-shop $ docker exec -it my-pg psql -U postgres -d kant_shop -c "SELECT * FROM product"
 id |     name      | price  | stock
----+---------------+--------+-------
  1 | 무선 이어폰   |  35000 |   100
  2 | 게이밍 마우스 |  28000 |    85
  3 | 27인치 모니터 | 259000 |    15
  4 | USB-C 허브    |  19900 |   200
  5 | 노트북 거치대 |  24000 |     7
(5 rows)
```

붕어빵(컨테이너)은 새로 찍었지만 보관함(pgdata)은 그대로라 상품 다섯 개가 다시 보여요.

4. 볼륨

쓰고 있는 볼륨은 지울 수 없어요

```
~/kant-shop $ docker volume rm pgdata
Error response from daemon: remove pgdata: volume is in use -
[1bb2985a53570708f7efc19cf0f1bd50e022dc9153fc14ce194908bf1cde8b62]

… docker stop my-pg, docker rm my-pg 뒤
~/kant-shop $ docker volume rm pgdata
pgdata
~/kant-shop $ docker volume ls
DRIVER    VOLUME NAME
```

-v 뒤에 오는 것 두 가지 (30에서 배웠어요)

`-v pgdata:/var/lib/postgresql`	**볼륨**	Docker가 관리하는 보관함, 데이터베이스 파일
`-v ./site:/usr/share/nginx/html`	**내 폴더**	내가 직접 열어 고치는 파일, 웹 페이지

5. YAML과 compose

YAML은 "이름: 값"을 한 줄에 하나씩 적는 설정 파일 형식이에요

compose.yaml

```yaml
services:
  db:
    image: postgres:18
    container_name: my-pg
    environment:
      POSTGRES_PASSWORD: admin1234
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql

volumes:
  pgdata:
```

- `services:` — 실행할 컨테이너(서비스) 목록, 꼭 있어야 해요
- `db:` — 서비스 이름, 직접 정해요 (db, web ...)
- `image: postgres:18` — 쓸 이미지:태그
- `container_name: my-pg` — 컨테이너 이름 (--name), 안 적으면 자동
- `environment:` — 환경 변수 (-e)
- `ports:` — 포트 연결 (-p), 따옴표로 감싸요
- `volumes:` — 볼륨 연결 (-v)
- `volumes:` — 쓸 볼륨 이름 적어 두기 (맨 바깥)

이름: 값 : 뒤에 한 칸 띄어요

- 값 목록의 한 칸

설명 주석, 실행 안 해요

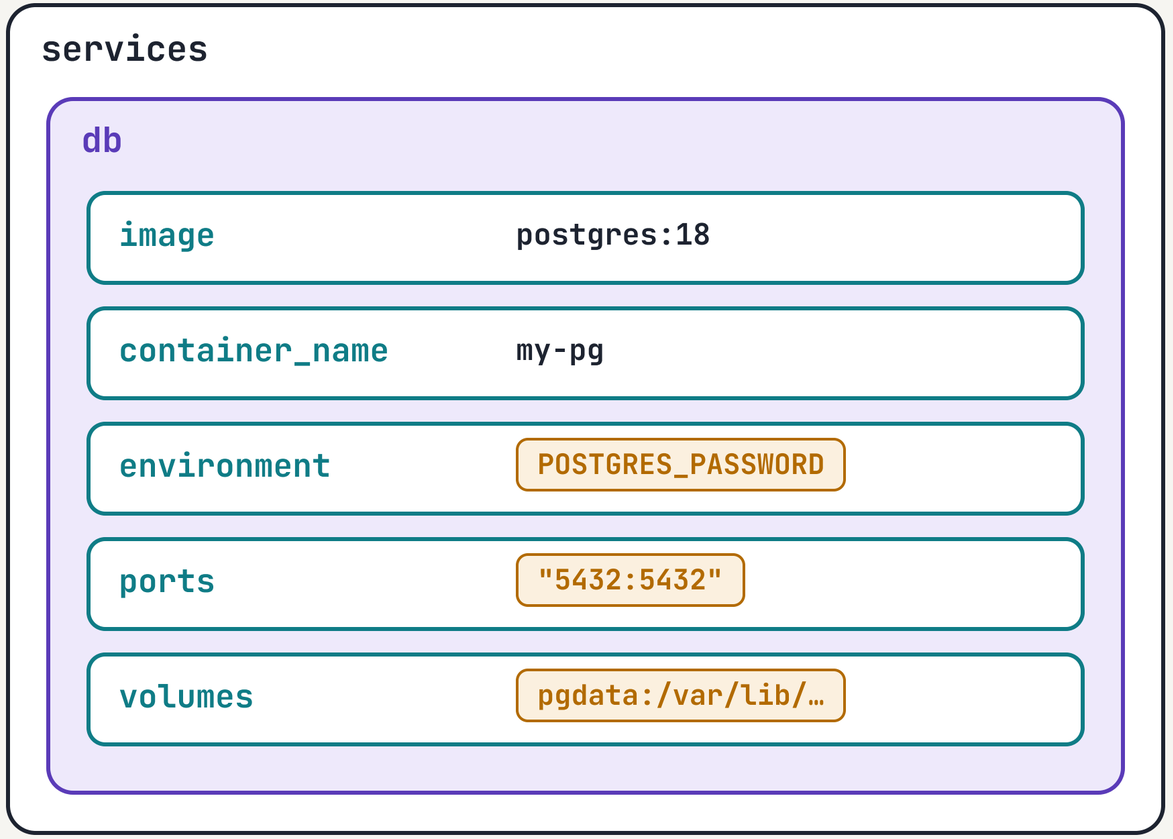
5. YAML과 compose

들여쓰기가 "누구 안에 있는지"를 정해요

compose.yaml

```yaml
services:
  db:
    image: postgres:18
    container_name: my-pg
    environment:
      POSTGRES_PASSWORD: admin1234
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql

volumes:
  pgdata:
```



한 단계 들어갈 때마다 띄어쓰기 2칸이에요. 같은 단계는 같은 칸에서 시작해요.

5. YAML과 compose

탭, 어긋난 칸, 쌍점 뒤 띄어쓰기에서 오류가 나요

```
2  db:
3  ⇥  image: postgres:18
4     container_name: my-pg
5     environment:
```

탭으로 들여썼어요

```
go-yaml load error in scanner (while scanning for the next token) at L3.C1: found character that cannot start any token
```

L3.C1 = 3번째 줄 1번째 칸에 탭

```
2  db:
3     image: postgres:18
4  container_name: my-pg
5     environment:
```

한 줄만 2칸이에요

```
go-yaml load error in scanner at L5.C16: mapping values are not allowed in this context
```

L5.C16 = 4번째 줄이 어긋나서 다음 줄에서 오류

```
2  db:
3     image:postgres:18
4     container_name: my-pg
5     environment:
```

image: 뒤에 띄어쓰기가 없어요

```
go-yaml load error in scanner at L4.C19: mapping values are not allowed in this context
```

L4.C19 = 위 줄이 값 하나로 읽혀서 다음 줄에서 오류

5. YAML과 compose

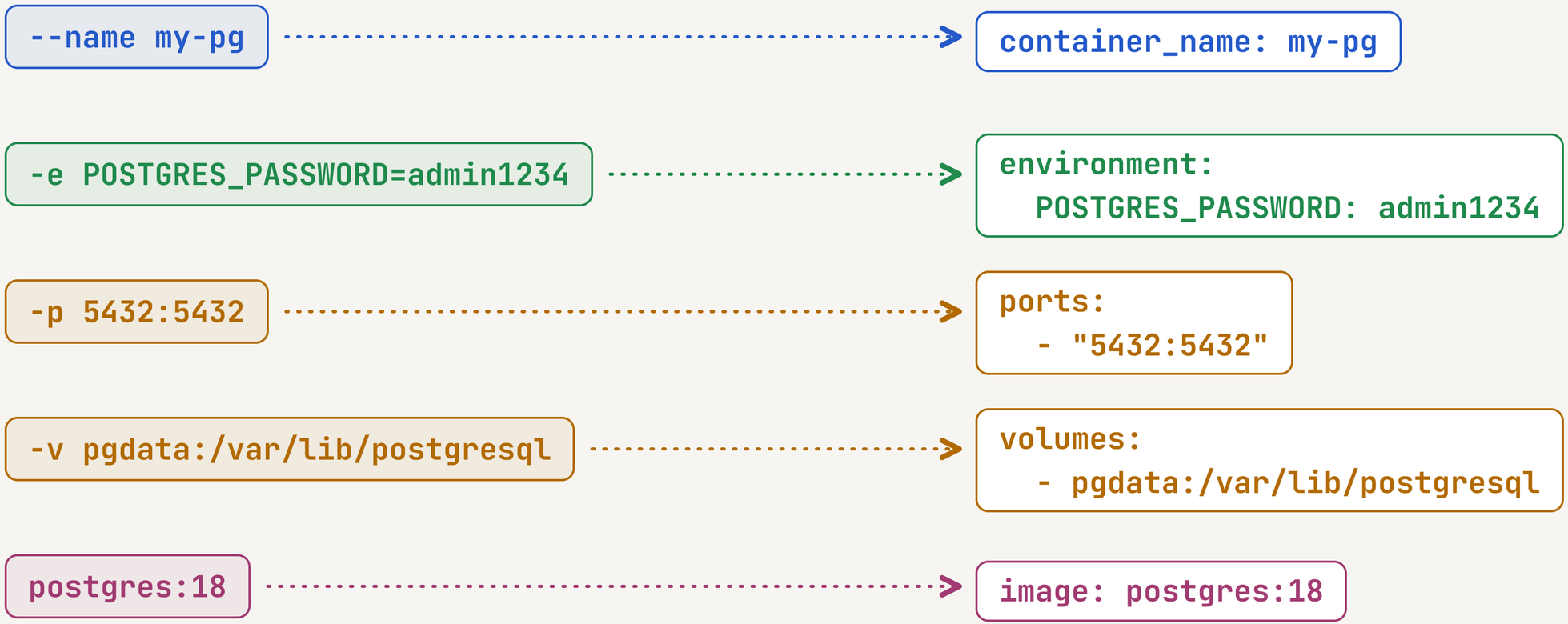
docker run의 옵션이 compose.yaml의 칸으로 옮겨 가요

docker run -d

한 줄 명령

services: db:

compose.yaml (db 안쪽만, 앞의 들여쓰기는 줄였어요)



-d는 docker compose up -d에 붙여요. 옵션 하나가 칸 하나예요.

5. YAML과 compose

compose config는 파일을 읽은 그대로 펼쳐 보여 줘요

```
~/kant-shop $ docker compose config
name: kant-shop
services:
  db:
    container_name: my-pg
    environment:
      POSTGRES_PASSWORD: admin1234
    image: postgres:18
    networks:
      default: null
    volumes:
      - type: volume
        source: pgdata
        target: /var/lib/postgresql
        volume: {}
    ports:
      - mode: ingress
        target: 5432
        published: "5432"
        protocol: tcp
networks:
  default:
    name: kant-shop_default
volumes:
  pgdata:
    name: kant-shop_pgdata
```

폴더 이름이 프로젝트 이름

내가 적은 칸

볼륨 이름 앞에 kant-shop_

up 하기 전에 쳐 봐요

파일에 오류가 있으면 여기서 오류가 나요. 오류가 없으면 Docker가 알아들은 내용을 빠짐없이 펼쳐 보여 줘요. 순서가 바뀌고 안 적은 칸이 채워져 있어도 괜찮아요.

5. YAML과 compose

up -d로 켜고 ps로 보고 exec로 들어가요

```
~/kant-shop $ docker compose up -d
[+] up 3/3
 ✔ Volume kant-shop_pgdata   Created                                0.0s
 ✔ Network kant-shop_default Created                                0.0s
 ✔ Container my-pg           Started                                0.2s
~/kant-shop $ docker compose ps
NAME      IMAGE         COMMAND                  SERVICE   CREATED         STATUS         PORTS
my-pg     postgres:18   "docker-entrypoint.s…"   db        3 seconds ago   Up 2 seconds   0.0.0.0:5432->5432/tcp
~/kant-shop $ docker exec -i my-pg psql -U postgres < kant_shop.sql
… 결과 6줄 줄임 (마지막 줄 INSERT 0 5)
~/kant-shop $ docker compose exec db psql -U postgres -d kant_shop -c "SELECT * FROM product"
 id |     name      | price  | stock
----+---------------+--------+-------
  1 | 무선 이어폰   |  35000 |   100
  2 | 게이밍 마우스 |  28000 |    85
  3 | 27인치 모니터 | 259000 |    15
  4 | USB-C 허브    |  19900 |   200
  5 | 노트북 거치대 |  24000 |     7
(5 rows)
```

네트워크, 볼륨, 컨테이너를 한 번에 만들어요

SERVICE 칸에 서비스 이름 db

컨테이너 이름 my-pg로도 돼요

compose exec 뒤에는 서비스 이름

logs, stop, start, restart 앞에 compose만 붙여요

```
~/kant-shop $ docker compose logs --tail 3
my-pg  | 2026-11-13 04:45:01.081 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
my-pg  | 2026-11-13 04:45:01.084 UTC [73] LOG:  database system was shut down at 2026-11-13 04:45:00 UTC
my-pg  | 2026-11-13 04:45:01.086 UTC [1] LOG:  database system is ready to accept connections
~/kant-shop $ docker compose logs -f --tail 2
my-pg  | 2026-11-13 04:45:01.084 UTC [73] LOG:  database system was shut down at 2026-11-13 04:45:00 UTC
my-pg  | 2026-11-13 04:45:01.086 UTC [1] LOG:  database system is ready to accept connections
^C
~/kant-shop $ docker compose stop
[+] stop 1/1
 ✔ Container my-pg Stopped                                                0.2s
~/kant-shop $ docker compose ps -a
NAME      IMAGE         COMMAND                  SERVICE   CREATED         STATUS                             PORTS
my-pg     postgres:18   "docker-entrypoint.s…"   db        5 seconds ago   Exited (0) Less than a second ago
~/kant-shop $ docker compose start
[+] start 1/1
 ✔ Container my-pg Started                                                0.1s
~/kant-shop $ docker compose restart
 Container my-pg Restarting
 Container my-pg Started
```

5. YAML과 compose

down은 컨테이너를 지우고 볼륨은 남겨요. down -v는 볼륨까지 지워요

```
~/kant-shop $ docker compose down
[+] down 2/2
 ✔ Container my-pg           Removed                                0.2s
 ✔ Network kant-shop_default Removed                                0.1s
~/kant-shop $ docker volume ls
DRIVER    VOLUME NAME
local     kant-shop_pgdata
~/kant-shop $ docker compose up -d
[+] up 2/2
 ✔ Network kant-shop_default Created                                0.0s
 ✔ Container my-pg           Started                                0.2s
~/kant-shop $ docker compose exec db psql -U postgres -d kant_shop -c "SELECT * FROM product"
 id |     name      | price  | stock
----+---------------+--------+-------
  1 | 무선 이어폰   |  35000 |   100
  2 | 게이밍 마우스 |  28000 |    85
… (5 rows)
~/kant-shop $ docker compose down -v
[+] down 3/3
 ✔ Container my-pg           Removed                                0.2s
 ✔ Network kant-shop_default Removed                                0.2s
 ✔ Volume kant-shop_pgdata   Removed                                0.1s
~/kant-shop $ docker volume ls
DRIVER    VOLUME NAME
```

down 뒤에도 볼륨이 남아요

다시 up 하면 상품이 그대로

-v는 볼륨까지 지워요

5. YAML과 compose

container_name을 빼면 이름이 "폴더-서비스-1"로 붙어요

```
~/kant-shop $ docker compose up -d
[+] up 3/3
 ✔ Network kant-shop_default Created                                    0.0s
 ✔ Volume kant-shop_pgdata   Created                                    0.0s
 ✔ Container kant-shop-db-1  Started                                    0.2s
~/kant-shop $ docker compose ps
NAME            IMAGE         COMMAND                  SERVICE   CREATED                  STATUS                  PORTS
kant-shop-db-1  postgres:18   "docker-entrypoint.s…"   db        Less than a second ago   Up Less than a second   0.0.0.0:5432->5432/tcp
~/kant-shop $ docker exec my-pg ls /tmp
Error response from daemon: No such container: my-pg
```

그 밖의 compose 명령

`docker compose ls`	compose로 켠 프로젝트 목록	`kant-shop running(1)`
`docker compose pull`	compose.yaml의 이미지를 미리 받기	`✔ Image postgres:18 Pulled`

6. 오류와 실제 상황

오늘 만난 오류 여덟 가지를 한 장에 모았어요

이런 글이 보이면	이렇게 해요
`Conflict. The container name "/my-pg" is already in use` 같은 이름의 컨테이너가 있어요	`docker rm my-pg` 뒤 다시
`Bind for 0.0.0.0:5432 failed: port is already allocated` 5432번을 다른 컨테이너가 써요	하나 끄거나 `-p 5433:5432`
`invalid reference format / -e: command not found` 줄 끝 \ 뒤에 띄어쓰기가 있어요	`\`로 줄을 끝내기
`cannot attach stdin to a TTY-enabled container` < 파일과 -it를 같이 썼어요	`-i`만 쓰기
`lstat …/kant_shop.sq: no such file or directory` 보낼 파일 이름이 틀렸거나 다른 폴더예요	`ls`로 이름 확인
`No such container: my-pg` container_name을 안 적어서 이름이 달라요	`docker compose ps`로 이름 보기
`volume is in use` 그 볼륨을 쓰는 컨테이너가 있어요	`stop`, `rm` 뒤 `volume rm`
`go-yaml load error … at L3.C1` YAML에 탭, 어긋난 칸, 쌍점 뒤 띄어쓰기	`docker compose config`로 확인

6. 오류와 실제 상황

예제 1. 수업 흐름 그대로: 켜고, 넣고, 실행하고, 확인해요

1. 켜기

```
docker run -d \
  --name my-pg \
  -e POSTGRES_PASSWORD=admin1234 \
  -p 5432:5432 postgres:18
```

2. 넣기

```
docker cp kant_shop.sql \
  my-pg:/tmp/kant_shop.sql
```

3. 실행하기

```
docker exec -it my-pg \
  psql -U postgres \
  -f /tmp/kant_shop.sql
```

4. 확인하기

```
docker exec -it my-pg \
  psql -U postgres -d kant_shop \
  -c "SELECT * FROM product"
```

```
~/kant-shop $ docker exec -it my-pg psql -U postgres -d kant_shop -c "SELECT * FROM product"
 id |      name       | price  | stock
----+-----------------+--------+-------
  1 | 무선 이어폰     |  35000 |   100
  2 | 게이밍 마우스   |  28000 |    85
  3 | 27인치 모니터   | 259000 |    15
  4 | USB-C 허브      |  19900 |   200
  5 | 노트북 거치대   |  24000 |     7
(5 rows)
```

2, 3 대신 docker exec -i my-pg psql -U postgres < kant_shop.sql 한 줄로 해도 돼요.

6. 오류와 실제 상황

예제 2. compose로 같은 일을 해요: 파일 두 개, 명령 세 줄

~/kant-shop

kant_shop.sql compose.yaml

```yaml
services:
  db:
    image: postgres:18
    container_name: my-pg
    environment:
      POSTGRES_PASSWORD: admin1234
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql

volumes:
  pgdata:
```

```
~/kant-shop $ docker compose up -d
[+] up 3/3
 ✔ Network kant-shop_default Created                                    0.0s
 ✔ Volume kant-shop_pgdata   Created                                    0.0s
 ✔ Container my-pg           Started                                    0.2s
~/kant-shop $ docker exec -i my-pg psql -U postgres < kant_shop.sql
DROP DATABASE
NOTICE:  database "kant_shop" does not exist, skipping
CREATE DATABASE
You are now connected to database "kant_shop" as user "postgres".
CREATE TABLE
INSERT 0 5
~/kant-shop $ docker exec -it my-pg psql -U postgres -d kant_shop -c "SELECT * FROM product"
 id |     name      | price  | stock
----+---------------+--------+-------
  1 | 무선 이어폰   |  35000 |   100
  2 | 게이밍 마우스 |  28000 |    85
  3 | 27인치 모니터 | 259000 |    15
  4 | USB-C 허브    |  19900 |   200
  5 | 노트북 거치대 |  24000 |     7
(5 rows)
```

내일은 docker compose up -d 한 줄이면 돼요. down만 했다면 상품도 그대로 있어요.

6. 오류와 실제 상황

예제 3. 다 쓴 것은 컨테이너, 볼륨, 이미지 순서로 지워요

```
~/kant-shop $ docker compose down -v
[+] down 3/3
 ✔ Container my-pg           Removed                                0.2s
 ✔ Volume kant-shop_pgdata   Removed                                0.1s
 ✔ Network kant-shop_default Removed                                0.2s
~/kant-shop $ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
~/kant-shop $ docker volume ls
DRIVER    VOLUME NAME
~/kant-shop $ docker rmi postgres:18
Untagged: postgres:18
Deleted: sha256:b9f6b8ff03b62f3cfd9d9ba3fe56226a9991c79d3fb46fd46536263bf02be553
~/kant-shop $ docker images
IMAGE   ID             DISK USAGE   CONTENT SIZE   EXTRA
```

빈 ps -a, 빈 volume ls, 빈 images가 다 지운 모습이에요.

compose로 켰다면

```
docker compose down -v
docker rmi postgres:18
```

docker run으로 켰다면

```
docker stop my-pg
docker rm my-pg
docker volume rm pgdata
docker rmi postgres:18
```

이미지는 다음 수업에 또 쓸 거라면 남겨 둬요. 지우면 다음에 pull부터 다시 해요.

정리

인터넷 자료에는 같은 명령의 긴 이름도 나와요

수업에서 쓰는 이름	같은 명령 (긴 이름)
`docker ps`	`docker container ls`
`docker images`	`docker image ls`
`docker rm my-pg`	`docker container rm my-pg`
`docker rmi postgres:18`	`docker image rm postgres:18`
`docker exec -it my-pg bash`	`docker container exec -it my-pg bash`
`docker cp …`	`docker container cp …`

```
~/kant-shop $ docker container ls
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS         PORTS                    NAMES
b64cc6816076   postgres:18   "docker-entrypoint.s…"   2 seconds ago   Up 2 seconds   0.0.0.0:5432->5432/tcp   my-pg
```

docker ps와 똑같은 표가 나와요. 어느 쪽을 쳐도 돼요.

정리

옵션이 기억 안 나면 명령 뒤에 --help를 붙여요

```
~/kant-shop $ docker cp --help
Usage:  docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH|-
        docker cp [OPTIONS] SRC_PATH|- CONTAINER:DEST_PATH

Copy files/folders between a container and the local filesystem

Use '-' as the source to read a tar archive from stdin
and extract it to a directory destination in a container.
Use '-' as the destination to stream a tar archive of a
container source to stdout.

Aliases:
  docker container cp, docker cp

Options:
  -a, --archive       Archive mode (copy all uid/gid information)
  -L, --follow-link   Always follow symlinks in SRC_PATH
  -q, --quiet         Suppress progress output during copy. Progress
                      output is automatically suppressed if no terminal
                      is attached
```

다시 볼 챕터

29

설치, pull, run, ps, stop, rm, logs

30

exec, 볼륨, compose 맛보기

31

compose 서비스 여러 개, logs

33 · 34

-e, MySQL, PostgreSQL

35

오늘 모은 명령, 치트시트

치트시트

docker 명령 한 장

이미지

`docker pull postgres:18`
Docker Hub에서 받기

`docker images`
받은 이미지 목록

`docker rmi postgres:18`
지우기 (쓰는 컨테이너 먼저 지우기)

컨테이너

```
docker run -d --name my-pg \
  -e POSTGRES_PASSWORD=admin1234 \
  -p 5432:5432 postgres:18
```
뒤에서 켜기, 이름, 환경 변수, 포트

`docker ps / docker ps -a`
켜진 것 / 꺼진 것까지

`docker stop my-pg`
끄기

`docker start my-pg`
다시 켜기

`docker restart my-pg`
껐다가 바로 켜기

`docker logs --tail 3 my-pg`
마지막 3줄 (-f는 계속, Ctrl+C)

`docker rm my-pg`
지우기 (끈 다음에)

안에서 작업

`docker exec -it my-pg bash`
들어가기, exit로 나오기

`docker exec my-pg ls /tmp`
명령 하나만 실행

```
docker exec -it my-pg \
  psql -U postgres -d kant_shop
```
psql 열기, \q로 나오기

```
docker cp kant_shop.sql \
  my-pg:/tmp/kant_shop.sql
```
내 파일을 컨테이너로

```
docker exec -i my-pg \
  psql -U postgres < kant_shop.sql
```
파일 내용 넘기기 (-it 말고 -i)

볼륨

`docker volume create pgdata`
만들기

`docker volume ls`
목록

`docker volume inspect pgdata`
자세히 보기

`-v pgdata:/var/lib/postgresql`
run에 붙여 연결

`docker volume rm pgdata`
지우기 (쓰는 컨테이너 먼저)

Windows Git Bash에서 docker cp, psql -f처럼 컨테이너 안 경로를 쓰기 전에 export MSYS_NO_PATHCONV=1을 먼저 쳐요.

치트시트

docker compose 명령 한 장

compose.yaml

```yaml
services:
  db:
    image: postgres:18
    container_name: my-pg
    environment:
      POSTGRES_PASSWORD: admin1234
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql

volumes:
  pgdata:
```

2칸 들여쓰기 · 쌍점 뒤 한 칸
container_name을 빼면 이름은 kant-shop-db-1

켜고 보기

`docker compose config`
파일을 읽은 그대로 보기, YAML 오류 확인

`docker compose up -d`
만들고 켜기 (뒤에서)

`docker compose ps`
이 폴더의 컨테이너

`docker compose ps -a`
꺼진 것까지

`docker compose ls`
compose 프로젝트 목록

`docker compose pull`
이미지 미리 받기

다루고 끄기

```
docker compose exec db \
  psql -U postgres -d kant_shop
```
서비스 이름 db로 들어가기

`docker compose logs --tail 3`
마지막 3줄 (-f는 계속)

`docker compose stop`
끄기 (지우지 않아요)

`docker compose start`
다시 켜기

`docker compose restart`
껐다가 바로 켜기

`docker compose down`
지우기, 볼륨은 남아요

`docker compose down -v`
볼륨까지 지우기

모든 compose 명령은 compose.yaml이 있는 폴더(~/kant-shop)에서 쳐요.